

HTML & CSS Answers — For Java Backend Developers

Concise answers to common HTML & CSS interview questions, with examples and Java backend integration tips.

Introduction

This guide provides concise, interview-ready answers to HTML and CSS questions with examples and context for Java backend developers. It also includes a short React overview and integration tips with Spring Boot.

HTML — Basic Questions

Q: What is HTML and what does it stand for?

A: HTML stands for HyperText Markup Language. It is the standard markup language used to create and structure content on the web (text, images, links, forms).

Q: Explain the difference between HTML and XHTML.

A: XHTML is a stricter, XML-based variant of HTML. XHTML requires well-formed XML (closed tags, lower-case, proper nesting). HTML5 relaxed many rules and is more forgiving; use HTML5 for modern web development.

Q: What are semantic tags? Give examples like `<header>`, `<article>`, `<section>`.

A: Semantic tags provide meaning to the document structure for browsers and assistive tech. Examples: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<footer>`. They improve accessibility and SEO.

Q: What is the difference between `<div>` and `<span>`?

A: `<div>` is a block-level element (starts on new line) used for layout; `<span>` is inline and used to style parts of text. Use semantic tags where possible instead of generic `<div>/<span>`.

Q: Why is the `<!DOCTYPE html>` declaration required?

A: The `<!DOCTYPE html>` declaration tells the browser to render the page in standards mode (HTML5). Without it, browsers may enter quirks mode and display inconsistent behavior.

Q: What's the difference between block and inline elements?

A: Block elements (e.g., `<div>`, `<p>`) take full width and start on a new line. Inline elements (e.g., `<span>`, `<a>`) only take necessary width and flow within a line.

Q: How can you make an image responsive in HTML?

A: Use CSS: `max-width:100%; height:auto;` on the `<img>` so it scales with its container. In modern layouts, use `srcset` and `sizes` for responsive images and performance.

Q: What's the use of the `alt` attribute in `<img>` tag?

A: The `alt` attribute provides alternative text for images — used by screen readers and when images fail to load. Essential for accessibility and SEO.

Q: Explain `<head>`, `<body>`, `<meta>`, and `<title>` tags.

A: `<head>` contains metadata and links to resources (CSS, scripts). `<meta>` tags define charset, viewport, and SEO info. `<title>` sets the page title shown in browser tabs. `<body>` contains visible content.

Q: What's the difference between `<b>` vs `<strong>` and `<i>` vs `<em>`?

A: `<b>/<i>` are presentational (`<b>` bold, `<i>` italic). `<strong>/<em>` are semantic and convey importance or emphasis — they help accessibility and can be styled similarly but carry meaning.

HTML — Intermediate & Advanced

Q: What are meta tags, and how do they help with SEO?

A: Meta tags like description, keywords, and viewport help search engines and control how pages are displayed on devices. The meta description improves search result snippets; viewport controls mobile scaling.

Q: Explain how the DOM is built by the browser.

A: The browser parses HTML into the DOM tree. It creates nodes for elements and text, then CSSOM is built from styles; together they form the render tree used to layout and paint the page.

Q: What's the difference between localStorage, sessionStorage, and cookies?

A: localStorage persists data with no expiration (per origin). sessionStorage persists per tab session. Cookies are sent to the server on requests and can have expiration; use cookies for auth tokens if server needs them.

Q: What is the use of <canvas> and <svg>?

A: <canvas> is a pixel-based drawing surface good for dynamic bitmaps (games). <svg> is vector-based markup, scalable and DOM-accessible for icons and charts.

Q: Explain the importance of accessibility (a11y) in HTML.

A: Accessibility ensures content is usable by people with disabilities. Use semantic tags, ARIA attributes, proper alt text, keyboard navigability, and color contrast to improve a11y.

Q: What are ARIA attributes, and when should they be used?

A: ARIA (Accessible Rich Internet Applications) attributes (role, aria-label, aria-hidden) add semantics for assistive tech when native semantics are insufficient. Prefer native elements over ARIA when possible.

Q: What is lazy loading for images and why is it useful?

A: Lazy loading defers image loading until they are near the viewport, improving initial load performance and reducing data usage. Use loading='lazy' or IntersectionObserver for control.

Q: Explain Server-Side Rendering (SSR) vs Client-Side Rendering (CSR).

A: SSR renders HTML on the server and sends it to the client (faster first paint, better SEO). CSR sends JS to render UI in browser (richer interactivity). Many apps use hybrid approaches (SSR + hydration).

Q: How can you optimize HTML for faster load times?

A: Minify HTML, inline critical CSS, defer non-critical JS, use efficient images (WebP, AVIF), use compression (gzip/br/), and reduce DOM depth/size.

Q: What's the difference between <link> and @import for loading CSS?

A: <link rel='stylesheet'> is preferred — it loads in parallel. @import inside CSS is slower and can block rendering; avoid @import for critical styles.

CSS — Basic Questions

Q: Explain the CSS Box Model.

A: Box Model: content, padding, border, margin. box-sizing controls whether width includes padding/border (border-box helps predictable sizing).

Q: What's the difference between margin and padding?

A: Padding is space inside the element between content and border; margin is space outside the border between elements.

Q: What are inline, internal, and external CSS?

A: Inline: style attribute on elements. Internal: `<style>` in HTML head. External: linked .css file. External CSS promotes reuse and caching.

Q: What's the difference between em, rem, px, and % units?

A: px = absolute pixels. em = relative to parent font-size. rem = relative to root font-size. % = relative to parent property (width, etc.). Use rem for consistent scalable typography.

Q: What is the difference between relative, absolute, fixed, and sticky positioning?

A: relative positions relative to normal flow. absolute positions relative to nearest positioned ancestor. fixed positions relative to viewport (stays on screen). sticky toggles between relative and fixed based on scroll.

Q: What is z-index and how does stacking context work?

A: z-index sets stacking order within a stacking context. A new stacking context is created by positioned elements with certain properties (opacity <1, transform, etc.). Understand contexts to debug z-index issues.

Q: What's the difference between display: none and visibility: hidden?

A: display:none removes element from layout flow (no space). visibility:hidden hides but reserves layout space.

Q: What is the difference between float and flex layout?

A: float was used for layouts historically but is not a layout system. Flexbox is a modern one-dimensional layout system for aligning items in rows/columns.

Q: How can you center a div vertically and horizontally?

A: Use flexbox: parent {display:flex; align-items:center; justify-content:center;} or use grid or transform trick for known sizes.

Q: What's the purpose of box-sizing: border-box?

A: border-box makes width include padding and border, which simplifies layout calculations and avoids surprises.

CSS — Intermediate & Advanced

Q: What is CSS specificity and how does it determine which style applies?

A: Specificity is a weight calculation (inline styles, IDs, classes/attributes/pseudo-classes, elements). Higher specificity wins. Use specificity and cascade rules to manage styles predictably.

Q: Explain pseudo-classes and pseudo-elements.

A: Pseudo-classes (e.g., `:hover`, `:focus`) target element states. Pseudo-elements (e.g., `::before`, `::after`) represent virtual elements for styling content.

Q: What are media queries and how do they help with responsiveness?

A: Media queries apply styles based on device features (width, resolution). Use breakpoints to adjust layout and typography across devices.

Q: What is Flexbox, and when would you prefer it over Grid?

A: Flexbox is for one-dimensional layouts (row or column). Use Grid for two-dimensional layouts (rows + columns). Many layouts use both together.

Q: How do you make a layout responsive across mobile and desktop?

A: Use fluid widths, relative units (% , `rem`), media queries, responsive images (`srcset`), and a mobile-first approach to progressively enhance design.

Q: Explain the difference between `nth-child()` and `nth-of-type()`.

A: `nth-child` counts among all child nodes; `nth-of-type` counts among siblings of the same element type. Use when selecting specific structured items.

Q: What are CSS variables (custom properties) and how are they useful?

A: CSS variables (e.g., `--primary: #0d6efd`) allow dynamic theming and reuse. They can be updated at runtime (useful for theming without rebuilds).

Q: What is BEM and why use it?

A: BEM (Block__Element--Modifier) is a naming convention that reduces CSS specificity conflicts and makes components modular and maintainable.

Q: How can you optimize CSS for better performance?

A: Minify and concatenate CSS, load critical CSS inline, defer non-critical CSS, avoid excessive selectors and deep specificity, and use caching headers.

Q: What is critical CSS, and how can it help improve performance?

A: Critical CSS is the minimal CSS needed to render above-the-fold content. Inlining it reduces render-blocking requests and speeds up first meaningful paint.

Practical / Scenario-Based Answers

Q: How can you make a sticky navbar that stays at the top while scrolling?

A: Use `position: sticky; top: 0;` on the navbar. Ensure the parent doesn't hide overflow; sticky works within its scroll container.

Q: Create a two-column layout using Flexbox.

A: Parent: `display: flex; gap: 16px;` Child widths: `flex: 1` for equal columns or use `flex-basis` for fixed ratio. Use media queries to stack on small screens.

Q: How to make a responsive card grid using CSS Grid?

A: Use `display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 16px;` This creates responsive cards that fit available width.

Q: How can you create a CSS-only dropdown menu?

A: Use `:hover` on parent to toggle visibility of submenu (`display/block`) or use `:focus-within` for accessibility. Prefer JS for complex interactions.

Q: How do you implement dark mode toggle using CSS variables?

A: Define color variables on `:root` and `.dark` class override them. Toggle `.dark` on `<html>` or `<body>` via JS; CSS updates automatically.

Q: How can you make an element scrollable only in one direction?

A: Use `overflow-x: auto; overflow-y: hidden;` (or vice versa) and set a fixed height/width to trigger scrolling.

Q: How do you add a transition effect when hovering over a button?

A: Use `transition: all 0.2s ease;` and change `transform` or `background` on `:hover` (e.g., `transform: translateY(-2px);`).

Q: How do you handle browser compatibility in CSS?

A: Use feature queries `@supports`, vendor prefixes for older features, fallbacks, and test in target browsers. Polyfills or progressive enhancement help.

Q: How does z-index interact with parent elements?

A: z-index is relative to stacking context; positioned children with z-index are contained by parent's stacking context. Creating a new context isolates z-index from siblings.

Q: What causes layout thrashing, and how can you avoid it?

A: Layout thrashing is caused by repeated reads/writes to layout-triggering properties (`offsetWidth`, `getComputedStyle`). Batch reads and writes, use `requestAnimationFrame`, and avoid forced synchronous layouts.

Integration with Java Backend — Practical Tips

Serve static files with Spring Boot

Place built frontend (React/HTML/CSS/JS) in `src/main/resources/static` so Spring Boot serves UI alongside APIs. Alternatively, deploy frontend separately and use CORS.

CORS and security

Configure CORS in Spring Boot (`WebMvcConfigurer` or `@CrossOrigin`). For production, restrict allowed origins and secure endpoints with Spring Security and tokens (JWT).

JSON vs server-rendered HTML

Prefer JSON APIs for SPA frontends (React). For simple pages, server-side templates (Thymeleaf) can render HTML on the server and reduce client complexity.

Handling authentication

Use secure cookies (`HttpOnly`, `Secure`, `SameSite`) or JWTs for stateless auth. For server-rendered apps, session-based auth with CSRF protection is common.

Error handling and validation

Perform input validation on backend. Return clear HTTP status codes and JSON error bodies; frontend should handle and show user-friendly messages.

React Overview for Java Backend Developers

Components and Props

Components are reusable UI units (functional or class). Props are read-only inputs, like method parameters in Java.

State and Lifecycle

State holds component-specific mutable data. `useEffect` and lifecycle methods manage side effects similar to lifecycle callbacks in backend frameworks.

Fetching data

Use `fetch` or `Axios` to call backend REST APIs. Handle loading and error states; prefer JSON APIs from Spring Boot.

Routing

React Router handles client-side routes. Server must return `index.html` for unknown routes in SPA setups (`historyApiFallback`).

Build and Deploy

Build frontend with `npm/yarn` (`npm run build`) and serve static files from Spring Boot or a CDN. Use environment variables for API endpoints.

Good luck with your interviews! — If you want, I can also generate a practice quiz or sample answers in flashcard format.